

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ESPECIALIDAD DE INGENIERÍA INFORMÁTICA



Proyecto de Diseño y Desarrollo de Software
[1INF54-0982]

Estándar de Programación en Java

Equipo: 6F

Integrantes:

- Gabriel Angel Quispe Cerron
- Roly Alvaro Huacre Samaniego
- Amira Paola Rodriguez Almora
- Diego Alfredo Carmona Chumbimuni

2026-2

Lima, 15 de septiembre

Historial de versiones

Fecha	Versión	Descripción	Autor
15/09/2026	1.0	Se ha redactado la primera versión de los estándares de programación java	Equipo 6F

Contenido

1. Introducción.....	4
2. Reglas de sintaxis y formato.....	4
2.1. Estructura general de carpetas y archivos.....	4
2.1.1. Backend (Java).....	4
2.2. Indentación y longitud de línea.....	5
2.3. Llaves y bloques.....	5
2.4. Imports y organización de archivos.....	5
2.5. Buenas prácticas en Spring Boot y Java moderno.....	5
3. Convenciones de nomenclatura.....	6
3.1. Nomenclatura específica de ALNS e IPSO.....	6
4. Manejo de excepciones.....	6
4.1. Jerarquía de excepciones.....	6
4.2. Manejo global.....	7
2.6. Uso de Anotaciones y Lombok.....	7
2.7. Validación de Datos (Bean Validation).....	7
2.8. Políticas de Logging y Trazabilidad.....	8
5. Optimización de estructuras de datos para ALNS e IPSO.....	8
5.1. Selección de colecciones.....	8
5.2. Minimización de asignaciones de memoria.....	8
5.3. Complejidad algorítmica.....	9
5.4. Concurrencia y Multihilo en Optimización.....	9
6. Referencias y recursos.....	9

1. Introducción

Este documento define los estándares de codificación en Java aplicables al backend del sistema SysMile, un planificador de rutas de entrega de paquetes construido sobre Spring Boot. El backend implementa, además de la lógica CRUD habitual, los algoritmos metaheurísticos de optimización de rutas ALNS (Adaptive Large Neighborhood Search) e IPSO (Improved Particle Swarm Optimization), por lo que este estándar dedica una sección específica a las prácticas de codificación que afectan el rendimiento de dichos algoritmos.

El cumplimiento de este estándar es obligatorio para todo el código Java del backend, incluyendo controladores, servicios, repositorios, modelos y los módulos de optimización.

2. Reglas de sintaxis y formato

2.1. Estructura general de carpetas y archivos

2.1.1. Backend (Java)

-

```
None
backend/
├─ src/
│   └─ main/
│       └─ java/com/sysmile/
│           ├── controller/
│           ├── service/
│           ├── repository/
│           ├── model/
│           ├── dto/
│           ├── config/
│           ├── exception/
│           ├── security/
│           └─ SysMileApplication.java
│       └─ resources/
│           ├── application.yml
│           └─ db/migration/
```

```
|   └─ test/java/com/sysmile/  
└─ pom.xml  
└─ .env
```

- **controller**: expone los endpoints REST y recibe las peticiones HTTP.
- **service**: contiene la lógica de negocio (cálculo de rutas, asignación de pedidos, etc.).
- **repository**: interfaces de acceso a datos (Spring Data JPA) sobre PostgreSQL.
- **model**: entidades JPA que representan las tablas de la base de datos.
- **dto**: objetos de transferencia de datos usados en la capa de API.
- **config**: configuración de Spring (seguridad, CORS, beans, etc.).
- **exception**: excepciones personalizadas y manejador global de errores.
- **security**: configuración de autenticación y autorización (JWT).
- **db/migration**: scripts de migración de base de datos (Flyway).

2.2. Indentación y longitud de línea

- Usar 4 espacios por nivel de indentación; no usar tabulaciones.
- Limitar las líneas a 120 caracteres; dividir expresiones largas manteniendo la alineación de operadores.
- Una sola declaración por línea; no encadenar múltiples sentencias con punto y coma.

2.3. Llaves y bloques

- Usar el estilo K&R: la llave de apertura va en la misma línea que la declaración.
- Usar siempre llaves en estructuras de control (if, for, while), incluso para bloques de una sola línea, para evitar errores al modificar el código.

Java

```
if (route.getStops().isEmpty()) {  
    throw new EmptyRouteException(route.getId());  
}
```

2.4. Imports y organización de archivos

- No usar imports con comodín (import java.util.*).
- Ordenar imports: paquetes de Java, luego librerías de terceros (Spring, Lombok, etc.), luego paquetes internos com.sysmile.*.
- Un archivo .java debe contener una única clase o interfaz pública, con el mismo nombre del archivo.

2.5. Buenas prácticas en Spring Boot y Java moderno

- **Inyección de dependencias:** Usar inyección por constructor exclusivamente; evitar la inyección por campo mediante `@Autowired` en los miembros de la clase.
- **Objetos de transferencia (DTOs):** Utilizar records de Java para los DTOs inmutables en la capa de API.
- **Manejo de nulidad:** Utilizar `Optional<T>` únicamente como tipo de retorno en servicios y repositorios para representar valores opcionales; no usarlo en parámetros de métodos ni en campos de entidades.
- **Documentación con JavaDoc:** Incluir JavaDoc descriptivo en todas las interfaces de servicio y repositorios públicos, especificando la intención, los parámetros (`@param`), los valores de retorno (`@return`) y las excepciones lanzadas (`@throws`).

3. Convenciones de nomenclatura

Se mantienen las convenciones generales definidas en el documento base de estándares del proyecto, reforzadas con las siguientes reglas específicas para los módulos de optimización:

Elemento	Convención	Ejemplo
Paquetes	minúsculas, sin guiones	com.sysmile.optimization.alns
Clases e interfaces	PascalCase, sustantivo	AlnsSolver, RouteRepository
Métodos y variables	camelCase, verbo de acción para métodos	calculateRoute(), totalDistance
Constantes	MAYÚSCULAS_CON_GUION_BAJO	MAX_ITERATIONS, DESTROY_RATE
Clases de algoritmos	sufijo del algoritmo que implementan	AlnsDestroyOperator, IpsoParticle
Enums	PascalCase para el tipo, MAYÚSCULAS para valores	RouteStatus.IN_PROGRESS

3.1. Nomenclatura específica de ALNS e IPSO

- Las clases de operadores de destrucción/reparación de ALNS deben llevar el sufijo `DestroyOperator` o `RepairOperator` (ej. `RandomRemovalDestroyOperator`, `GreedyInsertionRepairOperator`).
- Las clases relacionadas a IPSO deben usar el prefijo `Ipso` (ej. `IpsoParticle`, `IpsoSwarm`, `IpsoVelocityUpdater`).
- Los parámetros del algoritmo (tasa de destrucción, número de partículas, inercia, coeficientes cognitivo/social) deben declararse como constantes o en una clase de

configuración inmutable (AlnsParameters, IpsoParameters), nunca como valores mágicos dentro del código.

4. Manejo de excepciones

4.1. Jerarquía de excepciones

- Definir una excepción base del dominio (SysMileException) de la cual heredan las excepciones específicas.
- Crear excepciones específicas por caso de negocio: RouteNotFoundException, InfeasibleSolutionException, OptimizationTimeoutException, InvalidVehicleCapacityException.
- No capturar Exception ni Throwable de forma genérica salvo en el manejador global; capturar siempre el tipo más específico posible.

4.2. Manejo global

- Centralizar la traducción de excepciones a respuestas HTTP en un @ControllerAdvice (GlobalExceptionHandler).
- Los algoritmos de optimización deben lanzar excepciones no verificadas (unchecked) para no contaminar las firmas de métodos matemáticos con cláusulas throws; el error se traduce a HTTP únicamente en la capa de servicio/controlador.
- Registrar con SLF4J el contexto de la excepción (id de ruta, iteración del algoritmo, semilla aleatoria usada) para permitir reproducir fallas de optimización.

Java

```
public class InfeasibleSolutionException extends SysMileException
{
    public InfeasibleSolutionException(String routeId, int
iteration) {
        super("No se encontró solución factible para la ruta " +
routeId
            + " en la iteración " + iteration);
    }
}
```

- No exponer trazas de pila (stack traces) en las respuestas de la API; devolver únicamente un código y un mensaje controlado.

2.6. Uso de Anotaciones y Lombok

- **Entidades JPA:** Queda estrictamente prohibido el uso de las anotaciones @Data y @EqualsAndHashCode en las clases del paquete model (entidades de base de datos). El uso de estas en entidades JPA genera bucles infinitos en relaciones

bidireccionales (al evaluar toString() o hashCode()) y problemas severos de rendimiento. Utilizar únicamente @Getter y @Setter.

```
Java
@Service
@RequiredArgsConstructor
public class RouteService {
    private final RouteRepository routeRepository;
    private final AlnsSolver alnsSolver;
}
```

- **Inyección de Dependencias:** Para cumplir con la regla de inyección por constructor, se debe utilizar la anotación @RequiredArgsConstructor de Lombok en los servicios y controladores, declarando las dependencias inyectadas como variables private final.

2.7. Validación de Datos (Bean Validation)

- **Capa de API:** Todos los objetos de transferencia de datos en el paquete dto deben protegerse con anotaciones de validación estándar (ej. @NotNull, @NotBlank, @Positive, @Size) para asegurar la integridad de la petición antes de que alcance la capa de service.
- **Manejo de Errores de Validación:** Las excepciones generadas por fallos de validación en los controladores (específicamente MethodArgumentNotValidException al usar @Valid) deben ser interceptadas obligatoriamente en el GlobalExceptionHandler, devolviendo una respuesta HTTP 400 (Bad Request) con una lista estructurada de los campos inválidos.

```
Java
public record CreateRouteRequest(
    @NotNull(message = "El ID de vehículo es obligatorio")
    Long vehicleId,

    @Size(min = 1, message = "Debe incluir al menos un pedido")
    List<@NotNull Long> orderIds
) {}
```

2.8. Políticas de Logging y Trazabilidad

- **Uso exclusivo de SLF4J:** Queda absolutamente prohibido el uso de System.out.print, System.out.println o System.err.println. Todo registro de eventos debe realizarse a través de SLF4J (se recomienda el uso de la anotación @Slf4j de Lombok).

- **Niveles de severidad:** Se debe utilizar INFO para registrar eventos clave del flujo de negocio (ej. inicio de procesamiento de un lote, término exitoso de la generación de una ruta). Se reservarán los niveles DEBUG y TRACE exclusivamente para las trazas de ejecución interna de los algoritmos ALNS e IPSO (ej. evaluaciones de vecindarios, actualización de feromonas o partículas); estos niveles permanecerán apagados en el entorno de producción.

```
Java
@Slf4j
@Service
public class AlnsOptimizationService {
    public Route optimize(RouteRequest request) {
        log.info("Iniciando optimización ALNS para la ruta ID: {} ", request.getId());
        log.debug("Evaluando vecindario en iteración {} ", currentIteration);
    }
}
```

5. Optimización de estructuras de datos para ALNS e IPSO

Los algoritmos ALNS e IPSO se ejecutan en ciclos iterativos intensivos (miles de evaluaciones de vecindario o de partículas), por lo que las decisiones de estructura de datos impactan directamente el tiempo de respuesta del planificador. Se establecen las siguientes reglas:

5.1. Selección de colecciones

- Preferir ArrayList sobre LinkedList para representar la secuencia de paradas de una ruta, ya que el acceso indexado es la operación dominante en los operadores de vecindario.
- Usar arreglos primitivos (double[], int[]) en lugar de colecciones de tipos envueltos (List<Double>) para las matrices de distancias/tiempos y para los vectores de velocidad/posición de IPSO, evitando el costo de autoboxing en bucles críticos.
- Usar HashSet para verificar pertenencia de un pedido a una ruta (operación $O(1)$) en lugar de recorrer listas con contains().

5.2. Minimización de asignaciones de memoria

- Evitar crear nuevas instancias de listas o arreglos en cada iteración del algoritmo; reutilizar buffers preasignados cuando el tamaño del problema es constante durante la ejecución.

- Evitar el uso de streams de Java (Stream API) dentro de los bucles internos de evaluación de fitness/costo, dado su overhead de boxing y creación de objetos intermedios; reservarlos para código de preparación de datos, no para el núcleo del algoritmo.
- Implementar operaciones de copia de soluciones (clonado de una ruta o partícula) mediante copias superficiales o estructuras persistentes cuando sea posible, evitando serialización/deserialización costosa.

5.3. Complejidad algorítmica

- Documentar en JavaDoc la complejidad temporal esperada (notación Big-O) de cada operador de destrucción/reparación y de cada paso de actualización de partículas.
- Evitar operaciones $O(n^2)$ o peores dentro de bucles que ya son $O(\text{iteraciones} \times n)$; si son inevitables, justificarlas y acotar el tamaño de n mediante parámetros de configuración.
- Usar colas de prioridad (PriorityQueue) para los operadores de inserción voraz (greedy insertion) en lugar de ordenar listas completas en cada paso.

5.4. Concurrencia y Multihilo en Optimización

- Si se implementa ejecución en paralelo (mediante CompletableFuture, hilos virtuales o Parallel Streams) para acelerar la evaluación simultánea de múltiples vecindarios en ALNS o la actualización de enjambres en IPSO, se debe garantizar la seguridad de los hilos (thread-safety).
- **Inmutabilidad de estado compartido:** Cualquier objeto o estructura de datos compartida entre múltiples hilos de evaluación (como la matriz estática de distancias, los parámetros de configuración o la referencia a la "mejor solución global") debe ser estrictamente inmutable o, en su defecto, gestionarse exclusivamente a través de estructuras concurrentes (ej. variables AtomicReference, cerrojos de java.util.concurrent.locks). Queda prohibida la mutación de colecciones estándar sin sincronización en estos flujos.

Java

```
public class GlobalBestSolutionHolder {
    private final AtomicReference<Solution> globalBest = new
AtomicReference<>();

    public void updateIfBetter(Solution candidate) {
        globalBest.accumulateAndGet(candidate, (current, next) ->
            (next.getCost() < current.getCost()) ? next :
current);
    }
}
```

6. Referencias y recursos

Google Java Style Guide: <https://google.github.io/styleguide/javaguide.html>

Documentación oficial de Spring Boot: <https://spring.io/projects/spring-boot>

Effective Java (Joshua Bloch) — capítulos sobre estructuras de datos y rendimiento.